

A Task Scheduling Model and Algorithm for Cooperative Satellite-Ground On-Orbit Processing

IEEE Publication Technology Department

Abstract—The rapid growth of satellite data acquisition has created significant challenges in processing capacity and task scheduling. Due to limited computational resources, power constraints, and thermal limitations, satellites cannot support large remote sensing models. Instead, such high-computation tasks are offloaded to the ground, while smaller models are deployed onboard. This paper addresses the task scheduling problem in satellite scenarios by developing a cost model that considers thermal, latency, power, and colocation factors. A reward function is constructed based on this model, and reinforcement learning—specifically an enhanced DQN method—is applied to optimize task allocation between ground and onboard resources. The main contributions include: A cost modeling approach for satellite computing tasks, enabling quantitative evaluation of different allocation strategies. Use of DQN with a reward function derived from cost parameters to quickly determine near-optimal scheduling policies. Proposal of the DQN-RSF algorithm, which incorporates epsilon reset, selective network updates, and buffer reset mechanisms. Experiments show that DQN-RSF outperforms standard DQN in satellite task scheduling.

Index Terms—Satellite Computing, Remote Sensing, Learning

Small Model Modeling; Reinforcement Learning

1. 航天非常火，应用呈爆发式增长。
2. 现在的数据处理的架构(任务调度度)，缺陷(集中式处理，数据下传受限于时间窗口和可见星弧段，智能性不足)
3-1. 文献1针对xxx提出了xx方案，取得了xx效果。But，文献1未能xxxx
3-2. XXX方法近年来被提出，用于解决xx问题[2-8]。
XXXXXXXXXXXXXXXX,XXXXXXXXXXXXXXXX,XXXXXX
XXXXXX,XXXXXXXXXXXX.
4. the selected factors, 权威性

WITH
activ
systems are
required to p
tasks but als
exploration
efficiency of
requirements
has become

In recent years, rapid advancements in deep learning algorithms have led to outstanding performance in remote sensing image analysis. Deep learning-based image segmentation techniques can quickly identify various types of ground targets; object detection and tracking algorithms (e.g., YOLO [1] and Faster R-CNN [2]) have significantly improved in both speed and accuracy, enabling widespread application in remote sensing image interpretation; image super-resolution and restoration technologies (e.g., SRCNN [3] and ESRGAN [4]) effectively enhance the clarity of low-quality images and are employed in remote sensing image super-resolution; image

Manuscript created October, 2020; This work was developed by the IEEE Publication Technology Department. This work is distributed under the L^AT_EX Project Public License (LPPL) (<http://www.latex-project.org/>) version 1.3. A copy of the LPPL, version 1.3, is included in the base L^AT_EX documentation of all distributions of L^AT_EX released 2003/12/01 or later. The opinions expressed here are entirely that of the author. No warranty is expressed or implied. User assumes all risk.

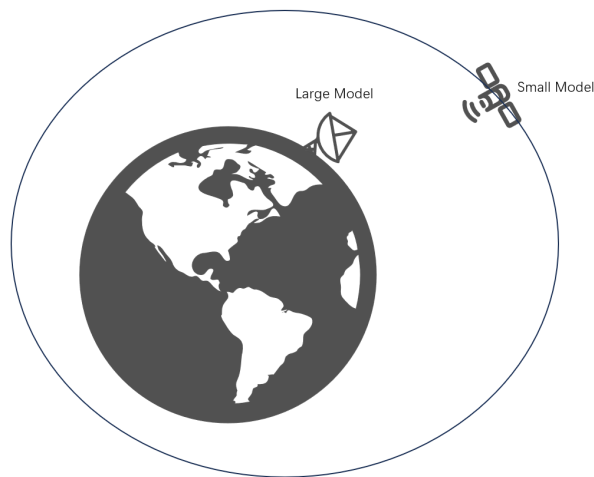


Fig. 1. Schematic diagram of the deployment of small and large remote sensing models across satellite and ground.

on and editing techniques based on Generative Adversarial Networks (GANs [5]), such as StyleGAN [6], have brought breakthroughs in generating realistic images in the remote sensing. These technological advances have improved the effectiveness and efficiency of image processing but have also brought revolutionary changes to the remote sensing industry, driving intelligent development.

Due to the explosive growth of deep learning algorithms, their application in remote sensing has yielded remarkable results. For instance, deep learning is widely used in areas such as remote sensing image segmentation and object recognition [7]. However, due to the unique constraints of the space environment, it is infeasible to deploy large-scale servers or computing clusters—commonly used on the ground—on satellites [8]. Satellites are limited to operating low-power, computationally constrained onboard hardware (e.g., the NVIDIA Jetson TX2 module), which typically provides only a fraction (e.g., one hundredth) of the computing power of GPUs commonly used for training networks, with power consumption limited to tens of watts. Consequently, large remote sensing models cannot be deployed directly on satellites; only ground-based computing resources meet the requirements of such models. The models deployed on satellite boards are typically small-scale, specialized models with limited parameters and computational demands—referred to as lightweight remote sensing models, as illustrated in Figure 1. This imposes high demands on satellite task scheduling capability: to conserve the satellite’s valuable energy resources, reduce heat generation during task execution, and ensure timely completion of

computational tasks, an intelligent task scheduling algorithm is needed to decide whether a task should be processed locally on the satellite or offloaded to the ground.

Therefore, this paper proposes an optimization scheme based on Reinforcement Learning (RL) algorithms [9]. The approach abstracts various constraints in satellite onboard computing and satellite-ground communication into distinct cost factors, forming a reward function accordingly. Through training, a model is developed to assist satellites in making effective task scheduling decisions. This model aims to reduce operational costs, conserve computational resources, and ensure the successful completion of tasks.

The development of a **quantitative computational cost analysis** model for satellite systems and its application in task scheduling decision-making hold significant theoretical and practical implications. Firstly, such a quantitative model provides a precise basis for cost evaluation in resource allocation, enabling optimal distribution of computational tasks. This helps prevent both resource wastage and system overload, substantially reducing operational expenses. Moreover, by comprehensively analyzing the cost and performance of individual satellite nodes, the model supports the formulation of more scientific and rational task allocation strategies. These strategies maximize system **efficiency** while **fulfilling** mission requirements, and help **identify which satellites operate most effectively under current loads**. Particularly for time-sensitive tasks, optimized scheduling decisions ensure timely and efficient processing, significantly enhancing the system's overall responsiveness and computational capability.

Secondly, the use of reinforcement learning (RL) for task scheduling allows for real-time dynamic optimization of allocation strategies, adapting effectively to the complexity and uncertainty inherent in satellite systems. Through continuous interaction with the environment, RL algorithms can identify **optimal or near-optimal scheduling** policies even under challenging conditions, thereby substantially improving overall system performance and reliability. Furthermore, **reinforcement learning enhances system fault tolerance**. In the event of unexpected incidents or urgent tasks, it can quickly generate efficient scheduling plans, ensuring both rational resource utilization and successful task completion. This dynamic adaptability not only increases the flexibility and robustness of satellite systems but also lays a solid foundation for future scalability and technological innovation.

The integration of quantitative modeling with reinforcement learning also promotes the transition toward **smarter and more autonomous satellite computing systems**. It provides strong support for achieving lower operational costs, higher efficiency, and improved service quality. By refining task scheduling strategies, higher computational efficiency and superior service outcomes can be attained within limited satellite resources, thereby increasing the return on investment. At the same time, the quantitative model offers objective metrics for evaluating different scheduling strategies, supporting organizations in making more economically sound decisions. This intelligent scheduling framework can also balance task requests from multiple users, ensuring consistent and stable service quality and enhancing the overall user experience.

In summary, proposing a **quantitative computational cost model** for satellite systems and employing reinforcement learning for task scheduling decisions can **significantly improve system efficiency and reliability, reduce costs, enhance fault tolerance**, and pave the way for future advancements. This combined approach drives satellite computing systems toward higher levels of intelligence and automation, aligning with global demands for sustainable, efficient, and autonomous technologies. It carries both immediate practical importance and long-term strategic value.

Currently, there is a notable lack of quantitative cost models specifically designed for satellite on-orbit computing. Among the various costs associated with satellite computation, research has predominantly focused on **latency**—that is, the delay between a ground station sending a request and the satellite (or satellite computing cluster) returning the result. In contrast, **other critical cost factors have received less attention, such as energy consumption, heat generation, and performance degradation when multiple computing tasks are co-located on the same satellite**. Given the stringent constraints on size, thermal management, and computational capacity in the space environment, it is essential to account for these cost **boundaries** during task scheduling. Thus, the field of **satellite edge computing** still lacks a comprehensive mathematical model that can quantify, integrate, and holistically optimize the various costs limiting satellite computational performance. Such a model would enable more rational task scheduling and offloading between satellites and ground stations, as well as among satellites themselves, thereby improving the efficiency and speed of computational tasks while extending satellite service life and reducing operational costs.

When coordinating and scheduling large and small remote sensing models in orbit, it is crucial to holistically consider factors such as thermal output, computational load, latency, co-location cost, and bandwidth usage. Effective management and optimization of these resources are directly linked to the system's operational efficiency and overall performance.

First, thermal management must be prioritized. Due to limited space and constrained heat dissipation capabilities onboard, high-performance computing operations—such as processing large volumes of remote sensing data—generate significant heat. This may lead to overheating, impairing functionality and shortening the satellite's lifespan. Therefore, intelligent thermal control strategies are essential to maintain operating temperatures within safe limits. Second, computational load directly influences hardware selection. Remote sensing tasks often involve processing high-resolution imagery and performing object detection, necessitating substantial computational power. It is important to balance performance requirements with power consumption and thermal constraints. Third, latency is critical, especially for real-time monitoring applications. The time taken from data acquisition to processing and final output directly affects system responsiveness. Co-location cost is another key consideration: when multiple computing tasks are assigned to the same onboard module, competition for resources can lead to significant delays. Thus, task allocation must be optimized to avoid congestion while maintaining efficiency. Finally, bandwidth usage directly im-

pacts data transmission efficiency. Remote sensing operations generate vast amounts of data, which must be transmitted over limited communication links to the ground or other satellites.

In summary, the coordination and scheduling of large and small remote sensing models require integrated consideration of thermal output, computational demand, latency, co-location cost, and bandwidth usage. Through carefully designed algorithms and strategic scheduling, it is possible to achieve efficient data processing and real-time responsiveness, meeting the stringent demands of modern remote sensing applications.

This paper addresses the task scheduling problem between large remote sensing models on the ground and small models onboard satellites. It proposes a quantitative mathematical model to evaluate the cost—i.e., resource consumption—associated with different scheduling strategies affecting computational resources in satellite systems. A reinforcement learning approach is employed to solve the formulated cost minimization problem, yielding a scheduling policy that effectively reduces operational overhead. The study provides a novel methodology for efficient utilization and cost-aware management of scarce computational resources in satellite environments.

The main **contributions** of this work are summarized as follows:

- 1) A cost evaluation framework for satellite computational tasks is established. The model incorporates key factors such as thermal impact, processing latency, energy consumption, and co-location overhead. This enables quantitative assessment of various task types and supports informed offloading decisions.
- 2) An enhanced algorithm named OSDQN-RSF is proposed, which introduces three key improvements: epsilon reset, selective updating of network parameters, and reset mechanisms for the experience replay buffer. The method significantly improves training accuracy and demonstrates broad applicability in reinforcement learning. The reward function is constructed using the cost model, with task assignment strategies as actions and task parameters as the state space. The trained neural network can rapidly schedule policies.

Preliminary Work

II. QUANTITATIVE COST MODELING FOR SATELLITE COMPUTATIONAL TASK PROCESSING

With the rapid development of satellite systems, particularly the proliferation of multi-satellite collaborations and the widespread application of high-resolution remote sensing data, the **complexity and scale of computational tasks** in space have significantly increased. However, due to the inherent limitations of satellite resources—**such as computational capacity, communication bandwidth, and energy supply**—efficiently allocating and scheduling these resources to meet diverse mission requirements has become a critical challenge. In this context, the motivation for quantitative cost modeling lies in providing a precise cost evaluation tool for satellite computational tasks, thereby optimizing resource allocation, improving system efficiency, and supporting scientifically informed decision-making.

TABLE I

KEY COST FACTORS IN SATELLITE COMPUTATIONAL TASK MODELING

Cost Factor	Rationale
Thermal Output	Strict thermal management is essential in space environments to maintain satellite functionality and prevent system failures.
Computational Load	Limited onboard computing capacity necessitates offloading computationally intensive tasks to ground stations.
Processing Latency	Time-sensitive remote sensing applications require strict adherence to real-time processing constraints.
Co-location Overhead	Concurrent execution of multiple tasks on a single satellite leads to resource contention and reduced efficiency.
Bandwidth Usage	Precious uplink/downlink capacity between satellite and ground requires careful allocation and conservation.

The primary objectives of quantitative cost modeling include: defining metrics to evaluate **resource consumption (e.g., computation time, energy usage, communication latency)**, assessing varying resource demands across different tasks, and enabling dynamic task scheduling and resource allocation through the model. It is also crucial to incorporate practical application scenarios to identify key influencing factors.

This chapter establishes a realistic cost model for satellite computational tasks based on parameters associated with various types of remote sensing operations. The model incorporates critical factors such as heat generation, required floating-point operations (FLOPs), latency, co-location cost, and bandwidth usage. Each factor is modeled within the action space of satellite task scheduling, ultimately forming a comprehensive mathematical framework for quantitative cost modeling in satellite computational processing. The specific factors considered are summarized in Table I.

A. Cost Factor Modeling

1) *Thermal Cost Factor Analysis*: The impact of heat generation on satellite onboard computing is multifaceted. First, excessive temperature may cause physical damage to electronic components, such as chip aging or failure, as well as damage to connectors and solder joints. Second, high temperatures reduce the efficiency of processors and other high-performance chips, leading to slower computation, decreased data processing capability, and longer task response time [10]. In addition, overheating may induce data errors in storage media or abnormal behavior in logic circuits, thus affecting data integrity and accuracy. Long-term heat accumulation can also shorten the lifespan of electronic components, increase system failure risk, and raise maintenance costs.

To mitigate overheating, satellites require additional power for cooling systems, which increases energy consumption, reduces battery life, and limits the available energy for other critical missions. Furthermore, high temperatures may increase electromagnetic noise, interfering with satellite communication systems and reducing the reliability and integrity of data

transmission. During the design stage, sufficient space and resources must be reserved for thermal dissipation systems, which may restrict the installation of other components or increase the overall weight. Addressing overheating issues usually requires complex solutions such as hardware replacement or software adjustment, which is nearly impossible for in-orbit maintenance.

Therefore, it is necessary to include heat generation in our cost modeling. This allows us to reasonably transfer high-heat tasks from onboard computation to ground-based computation during task scheduling.

The **mathematical** expression is as follows:

$$C_1 = \begin{cases} \rho \cdot \omega \cdot \tan\left(\frac{\pi}{2} \cdot \frac{Q}{Q_{\text{MAX}}}\right), & Q < Q_{\text{MAX}} \\ +\infty, & Q \geq Q_{\text{MAX}} \end{cases} \quad (1)$$

Here, C_1 represents the portion of the total cost related to heat generation; ρ is a dimensionless coefficient, since the total cost involves different physical quantities with varying units, requiring a normalization factor to unify them. ω is a weight, representing the relative importance of heat generation compared to other costs: if ω is larger, the model places greater emphasis on heat-related constraints, thus preferring scheduling schemes with lower heat. Q denotes the heat generated during computational tasks, measured in joules (J); Q_{MAX} refers to the thermal threshold (“red line”) of the satellite during operation.

Since satellite heat generation must be strictly controlled below this red line, we impose a penalty in the model when the generated heat approaches or exceeds the threshold. The penalty grows as the proportion of heat to the maximum allowable value increases, reaching infinity when $Q/Q_{\text{MAX}} = 1$. Hence, the tangent function is used to model this relationship, ensuring that C_1 becomes infinite at the threshold, which reflects the strict thermal constraints in satellite cost modeling.

2) *Computational Load Cost Factor Analysis:* Excessive computational workload in satellite onboard computing (e.g., a large number of floating-point operations) can exert multiple negative impacts on satellite performance and operation. First, a heavy floating-point workload substantially increases processor utilization, which not only consumes more power but also generates more heat. Elevated temperatures reduce the efficiency of electronic components and may even cause chip damage or shorten their lifespan. Moreover, computational overload can degrade system responsiveness and stability, potentially leading to data processing delays or errors, thereby undermining the satellite’s task execution capability.

From an energy perspective, high computational loads significantly increase power consumption, which poses a serious challenge for satellites relying on limited battery power and may restrict the operation of other critical functions. Finally, if computational tasks cannot be completed in a timely manner, this may result in uneven allocation of system resources, software crashes, or other failures, further threatening the overall reliability of the satellite. Therefore, when designing onboard computing tasks, it is essential to carefully balance

the computational workload with system capacity to ensure efficient and reliable task execution.

Consequently, we incorporate computational workload (measured in floating-point operations) into our cost modeling framework. This enables us to reasonably offload tasks with excessive onboard computational demand to ground-based systems during task scheduling.

The mathematical formulation is given as follows:

$$C_2 = \rho \cdot \omega \cdot \text{FLOPs} \quad (2)$$

Here, C_2 denotes the portion of the total cost related to computational workload; ρ and ω have the same meanings as in Equation (1); and FLOPs represents the floating-point operations required by a task, measured in mega floating-point operations (MFLOPs).

3) *Processing Latency Cost Factor Analysis:* In satellite onboard computing, time cost (including task processing delay and data transmission delay) has profound impacts on system performance, efficiency, and reliability.

First, task processing delay refers to the time required to complete a specific computational task. Such delay directly affects the satellite’s task execution capability: excessive task processing delay may lead to resource contention, reducing the efficiency of scheduling and execution for multiple tasks. Moreover, task backlog can lower the task completion rate, and in severe cases, cause task timeout or task abandonment. Since ground control expects the onboard task processing delay to be minimized, it is necessary to incorporate this factor into our cost model.

Second, data transmission delay refers to the time required for transmitting data from the satellite back to the ground control station. This delay also critically influences onboard computing: for tasks requiring real-time processing and feedback (such as autonomous navigation or emergency communications), excessive data transmission delay may result in untimely responses, or even missing the optimal window for task execution.

When scheduling a computational task, the satellite must decide whether to process it onboard or offload the raw data to the ground for computation. Onboard processing avoids the raw data transmission delay and only requires transmitting the final results back to the ground; however, limited onboard computational resources may cause relatively large task processing delays. By contrast, offloading the task to the ground avoids the heavy onboard workload but incurs significant data transmission delay.

These two sources of delay are represented in the total cost model as follows:

$$C_3 = \rho \cdot (\omega_1 \cdot t_1 + \omega_2 \cdot t_2) \quad (3)$$

Here, C_3 denotes the portion of the total cost associated with time; ρ has the same meaning as in Equation (1); ω_1 and ω_2 represent the weights of task processing delay and data transmission delay, respectively; t_1 and t_2 correspond to task processing delay and data transmission delay, respectively, measured in milliseconds (ms).

4) *Co-location Overhead Cost Factor Analysis*: In satellite onboard computing, co-location cost refers to the phenomenon whereby the efficiency of individual task processing decreases when multiple tasks run concurrently on the same satellite, due to resource competition, priority conflicts, and increased system complexity. This phenomenon has profound impacts on the overall satellite performance and task success rate.

Under limited resources, simultaneous execution of multiple tasks leads to the division of critical resources such as computing (e.g., processor time), storage (e.g., memory or data storage space), and communication bandwidth. For example, when a satellite must simultaneously perform remote sensing data processing, communication relay, and navigation positioning, each task requires a portion of the computational and storage resources. However, satellite hardware resources are typically fixed and constrained by strict weight, volume, and power limitations. As a result, coexisting tasks reduce the resources available to each individual task, thereby lowering its processing efficiency.

To address these challenges, co-location cost must be incorporated into the cost quantification model during task allocation.

The mathematical formulation is expressed as:

$$C_4 = \rho \cdot \omega \cdot f(n) \quad (4)$$

Here, C_4 denotes the portion of the total cost associated with co-location; ρ and ω have the same meanings as in Equation (1); and $f(n)$ represents the function describing how co-location cost increases with the number of tasks n being executed simultaneously on the satellite.

5) *Bandwidth Usage Cost Factor Analysis*: The bandwidth occupancy cost of satellite-to-ground data transmission has multiple impacts on onboard computing. First, onboard computing requires the processing and transmission of large volumes of data, including remote sensing images, communication data, and other mission-generated information. However, the satellite-to-ground communication link is strictly constrained by spectrum resources, and the available bandwidth is usually limited, which restricts data transmission rates. Such bandwidth limitations directly affect the efficiency of data processing in onboard computing, especially for tasks requiring real-time transmission or rapid response, where data backlog may significantly reduce overall system performance.

Second, bandwidth occupancy cost is also closely related to energy consumption. Satellite communication equipment requires electrical power to maintain data transmission, while the available energy is limited to what can be provided by solar panels and storage batteries. Onboard computing tasks must balance energy allocation between data processing and data transmission; excessive bandwidth usage may lead to insufficient energy for other critical missions (e.g., navigation, telemetry), thereby undermining the overall stability of satellite operations. Hence, the bandwidth occupancy cost directly influences data transmission efficiency, energy consumption, and task timeliness, exerting an important impact on the overall performance and applicability of onboard computing.

Therefore, it is necessary to incorporate bandwidth occupancy into the cost model during task scheduling.

The mathematical formulation is expressed as follows:

$$C_5 = \begin{cases} \rho \cdot \omega \cdot \tan\left(\frac{\pi}{2} \cdot \frac{B}{B_{\text{MAX}}}\right), & B < B_{\text{MAX}} \\ +\infty, & B \geq B_{\text{MAX}} \end{cases} \quad (5)$$

Here, C_5 denotes the portion of the total cost associated with bandwidth occupancy; ρ and ω have the same meanings as in Equation (1); B represents the bandwidth required for the downlink transmission of the scheduled task, measured in megabits per second (Mbps); and B_{MAX} is the maximum available downlink bandwidth of the satellite.

Since satellite transmission bandwidth is a scarce resource, bandwidth usage should be penalized in the model. The penalty grows as the ratio B/B_{MAX} increases, and approaches infinity when the ratio reaches unity. Therefore, the tangent function is employed to characterize this cost relationship, ensuring that C_5 becomes infinite when $B = B_{\text{MAX}}$, which reflects the strict bandwidth constraints in satellite cost modeling.

B. Cost Modeling in the *Action Space*

Considering different task types and characteristics, the action space of satellite task scheduling can be categorized into three primary modes:

- 1) Tasks are processed onboard the satellite using a lightweight remote sensing model, and only the results are transmitted back to the ground.
- 2) Task data are pre-processed and segmented onboard the satellite using a lightweight remote sensing model (suitable for tasks such as object recognition or disaster detection, where large irrelevant portions of the image do not need to be transmitted). The segmented data are then transmitted to the ground for further processing by a large-scale remote sensing model.
- 3) All required data are directly transmitted from the satellite to the ground, where a large-scale remote sensing model performs the complete processing.

This section develops cost models corresponding to these three different scheduling strategies, providing the mathematical foundation for applying reinforcement learning methods to task scheduling in the following chapter.

1) *Cost Modeling on the Satellite Side*: When a task is processed onboard the satellite using a lightweight remote sensing model, the total cost consists of the following components:

- heat generation cost during task processing,
- computational workload cost measured in floating-point operations,
- task processing time cost on the satellite,
- data transmission delay cost for returning the processed results to the ground,
- co-location cost when multiple tasks are processed simultaneously on the same satellite,
- bandwidth occupancy cost resulting from transmitting the task results to the ground.

Accordingly, the total cost in this scenario is modeled as:

$$\begin{aligned} \text{Cost}_s = & \rho_{\text{heat}}\omega_{\text{heat}} \tan\left(\frac{\pi}{2} \cdot \frac{Q}{Q_{\text{MAX}}}\right) + \rho_{\text{comp}}\omega_{\text{comp}} \text{FLOPs} \\ & + \rho_{\text{time}}(\omega_{\text{proc}}t_1 + \omega_{\text{trans}}t_2) + \rho_{\text{coloc}}\omega_{\text{coloc}} f(n) \\ & + \rho_{\text{bw}}\omega_{\text{bw}} \tan\left(\frac{\pi}{2} \cdot \frac{B}{B_{\text{MAX}}}\right). \end{aligned} \quad (6)$$

Here, Cost_s denotes the total cost of processing the task onboard the satellite; Q and Q_{MAX} are the actual and maximum allowable heat generation, respectively; FLOPs represents the computational workload in mega floating-point operations (MFLOPs); t_1 and t_2 correspond to task processing delay and result transmission delay, respectively; $f(n)$ denotes the co-location cost as a function of the number of simultaneously processed tasks n ; B and B_{MAX} denote the required and maximum available downlink bandwidth, respectively; ρ is a dimensionless coefficient for normalization, and ω , ω_{proc} , and ω_{trans} represent the weights associated with each cost component.

2) *Ground-Side Cost Modeling*: When tasks are processed on the ground by large-scale remote sensing models, the primary costs mainly consist of the following components:

- the thermal cost associated with downlink data transmission from the satellite to the ground,
- the latency cost incurred during transmission,
- the bandwidth occupation cost.

Since the ground has practically unlimited computational resources compared to the satellite, and the computational cost on the ground is not the main focus of this work, the ground-side computation cost is not considered here.

Accordingly, the cost in this scenario can be expressed as:

$$\begin{aligned} \text{Cost}_g = & \rho_{\text{heat}}\omega_{\text{heat}} \tan\left(\frac{\pi}{2} \cdot \frac{Q}{Q_{\text{MAX}}}\right) \\ & + \rho_{\text{time}}\omega_{\text{trans}} t_1 \\ & + \rho_{\text{bw}}\omega_{\text{bw}} \tan\left(\frac{\pi}{2} \cdot \frac{B}{B_{\text{MAX}}}\right), \end{aligned} \quad (7)$$

3) *Satellite-Ground Collaborative Cost Modeling*: When a task is preprocessed onboard the satellite by a lightweight remote sensing model (e.g., image segmentation) and then transmitted to a large-scale ground model for further processing, the total cost consists of the following components:

- heat generation cost during onboard preprocessing,
- time cost for preprocessing onboard,
- computational workload cost (FLOPs) for onboard preprocessing,
- transmission delay for sending the preprocessed data to the ground,
- heat generation cost associated with data transmission from the satellite to the ground,
- bandwidth occupancy cost for downlink transmission.

Accordingly, the total cost in this scenario can be expressed as:

$$\begin{aligned} \text{Cost}_{s\&g} = & \rho_{\text{heat}}\omega_{\text{heat}} \tan\left(\frac{\pi}{2} \frac{Q_{\text{pre}} + Q_{\text{trans}}}{Q_{\text{MAX}}}\right) \\ & + \rho_t(\omega_{\text{pre}}t_{\text{pre}} + \omega_{\text{trans}}t_{\text{trans}}) \\ & + \rho_{\text{comp}}\omega_{\text{comp}} \text{FLOPs} \\ & + \rho_{\text{bw}}\omega_{\text{bw}} \tan\left(\frac{\pi}{2} \frac{B}{B_{\text{MAX}}}\right) \end{aligned} \quad (8)$$

C. Summary

This chapter focuses on the quantitative cost modeling of satellite computational task processing. It primarily investigates how cost modeling can be employed to optimize the allocation and scheduling of satellite resources, addressing the challenges posed by the increasing complexity and limited availability of satellite systems. With the widespread adoption of multi-satellite collaborative operations and high-resolution remote sensing data, both the complexity and scale of satellite computational tasks are expanding, while resources such as computational power, communication bandwidth, and energy supply remain constrained. Efficient allocation and scheduling of these resources thus constitute a critical challenge. The objective of quantitative cost modeling is to provide precise tools for evaluating the costs associated with satellite computational tasks, thereby enabling optimized resource allocation, improved system efficiency, and informed decision-making.

During the modeling process, key factors including thermal cost, computational workload cost, time cost, co-location cost, and bandwidth occupancy cost were analyzed in detail. Thermal cost modeling considers issues such as electronic component damage, efficiency degradation, and data errors caused by high temperatures during onboard computation, and employs a tangent function to describe the relationship between thermal cost and the allowable heat budget. Computational workload cost modeling focuses on the impact of the number of floating-point operations required by tasks on satellite performance and energy consumption. Time cost modeling includes both task processing delay and data transmission delay, which affect task execution efficiency and real-time performance, respectively. Co-location cost modeling analyzes the efficiency reduction caused by resource contention when multiple tasks run concurrently. Bandwidth occupancy cost modeling addresses the impact of limited satellite downlink bandwidth on task scheduling.

Finally, this chapter develops cost models for three distinct task scheduling strategies: tasks processed entirely onboard the satellite, tasks processed on the ground, and tasks preprocessed onboard and then transmitted to the ground for further processing. These models provide the mathematical foundation for applying reinforcement learning methods to satellite task scheduling in subsequent chapters.

III. ALGORITHM DESIGN

Since the satellite computational task scheduling problem in this study aims to minimize the total cost (e.g., thermal, latency, etc.) by selecting the optimal strategy, its essence lies in choosing the optimal action (i.e., scheduling policy)

from a multidimensional action space for each state tuple (e.g., thermal, latency, etc.) in a multidimensional state space, such that the corresponding cost is minimized. Given the characteristics of satellite computational task scheduling, this study adopts reinforcement learning as the solution methodology. Reinforcement learning is particularly suitable for decision-making tasks in dynamic environments, where feedback is obtained through a reward mechanism. Therefore, it provides an appropriate and effective framework for the scheduling problem addressed in this research.

This chapter builds upon the quantitative cost modeling of satellite computational tasks presented in Chapter II, and focuses on **modeling the state space, action space, and reward function** of the algorithm employed in this study. Furthermore, the chapter introduces the **network architecture** and related design considerations used for implementing the proposed approach.

A. Modeling of State Space, Action Space, and Reward Function

In Chapter II, the various costs associated with satellite computational tasks were mathematically modeled, and the functional relationships between each cost component and the total cost were explicitly defined. Building upon this quantitative cost modeling, this section defines the state space, action space, and reward function of the algorithm employed in this study.

TABLE II

SUMMARY OF STATES IN THE STATE SPACE

Scheduling Strategy	State Symbol	State Description
Task processed on the satellite	Q_s	Heat cost
	$FLOPs_s$	Computational cost (floating-point operations)
	T_{s1}	Task processing latency
	T_{s2}	Result transmission latency
	$f(n)_s$	Co-location cost (when multiple tasks run simultaneously)
Task processed on the ground	B_s	Bandwidth occupation cost
	Q_g	Heat cost
	T_g	Data transmission latency
	B_g	Bandwidth occupation cost
Task pre-processed on the satellite and further processed on the ground	$Q_{pre_s\&g}$	Heat cost during preprocessing
	$Q_{trans_s\&g}$	Heat cost during transmission
	$T_{pre_s\&g}$	Preprocessing latency
	$T_{trans_s\&g}$	Data transmission latency
	$FLOPs_{s\&g}$	Computational cost (floating-point operations)
	$B_{s\&g}$	Bandwidth occupation cost

1) *State Space*: The state space serves as the basis for decision-making of reinforcement learning, which in this study corresponds to the various cost factors (e.g., heat, latency, etc.) defined in Chapter II. Since the three scheduling strategies for satellite computational tasks involve distinct heat, latency, and other cost components, these must be represented as different states. Based on the cost factors outlined in Section 2, the states defined in the state space are summarized in Table II.

In total, the state space comprises fifteen distinct states with continuous values. Accordingly, the state space can be characterized as fifteen-dimensional and continuous.

2) *Action Space*: The action space of the algorithm corresponds to the three satellite computing task scheduling strategies described in Chapter II: (1) processing the task with the onboard lightweight remote sensing model, (2) processing the task with the ground-based large-scale remote sensing model, and (3) pre-processing the task with the onboard lightweight model and subsequently transmitting it to the ground-based large model for computation. These three strategies constitute the three actions within the action space.

3) *Reward Function*: The reward function corresponding to each action of the algorithm is defined as the negative of the cost derived from the cost quantification model introduced in Chapter II. This formulation is adopted because the objective is to select the action that minimizes the overall cost. By taking the negative cost as the reward, the algorithm ensures that higher costs lead to lower rewards, thereby aligning the decision-making process with the goal of cost minimization.

Specifically, the reward function for processing a task using the onboard lightweight remote sensing model is defined as follows:

$$\begin{aligned} \text{reward}_s = -\text{Cost}_s = & -(\rho_{\text{heat}}\omega_{\text{heat}} \tan\left(\frac{\pi}{2} \cdot \frac{Q}{Q_{\text{MAX}}}\right) \\ & + \rho_{\text{comp}}\omega_{\text{comp}} \text{FLOPs} \\ & + \rho_{\text{time}}(\omega_{\text{proc}}t_1 + \omega_{\text{trans}}t_2) \\ & + \rho_{\text{coloc}}\omega_{\text{coloc}} f(n) \\ & + \rho_{\text{bw}}\omega_{\text{bw}} \tan\left(\frac{\pi}{2} \cdot \frac{B}{B_{\text{MAX}}}\right)). \end{aligned} \quad (9)$$

The reward function for processing a task using the ground-based large-scale remote sensing model is defined as:

$$\begin{aligned} \text{reward}_g = -\text{Cost}_g = & -(\rho_{\text{heat}}\omega_{\text{heat}} \tan\left(\frac{\pi}{2} \cdot \frac{Q}{Q_{\text{MAX}}}\right) \\ & + \rho_{\text{time}}\omega_{\text{trans}} t_1 \\ & + \rho_{\text{bw}}\omega_{\text{bw}} \tan\left(\frac{\pi}{2} \cdot \frac{B}{B_{\text{MAX}}}\right)), \end{aligned} \quad (10)$$

Finally, the reward function for pre-processing a task with the onboard lightweight model and subsequently transmitting it to the ground-based large-scale model for computation is defined as:

$$\begin{aligned} \text{reward}_{s\&g} = -\text{Cost}_{s\&g} = & -(\rho_{\text{heat}}\omega_{\text{heat}} \tan\left(\frac{\pi}{2} \cdot \frac{Q_{\text{pre}} + Q_{\text{trans}}}{Q_{\text{MAX}}}\right) \\ & + \rho_t(\omega_{\text{pre}}t_{\text{pre}} + \omega_{\text{trans}}t_{\text{trans}}) \\ & + \rho_{\text{comp}}\omega_{\text{comp}} \text{FLOPs} \\ & + \rho_{\text{bw}}\omega_{\text{bw}} \tan\left(\frac{\pi}{2} \cdot \frac{B}{B_{\text{MAX}}}\right)) \end{aligned} \quad (11)$$

The meanings of the variables appearing in these three reward functions are consistent with the explanations provided in Chapter II. At this point, the reward functions corresponding to the three actions in the action space are fully defined.

B. Network Architecture and Design Considerations

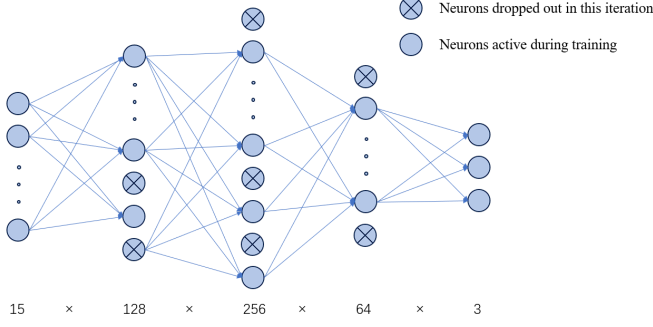


Fig. 2. Network architecture design, where neurons marked with a black “x” are dropped out in the current iteration, and unmarked neurons remain active during training.

Considering that the state space in this study consists of fifteen continuous-valued variables, a relatively deep network with a larger number of parameters was designed, as illustrated in Figure 2, to adequately capture the complex mappings during training.

In addition to fully connected layers, a Dropout layer [11] is introduced. During training, Dropout randomly masks a subset of neuron outputs (represented as hidden neurons in Figure 2), forcing the network to rely on a wider set of features rather than over-relying on specific neurons. This mechanism helps prevent overfitting and enhances the generalization capability of the model.

C. One-State DQN with Epsilon Reset, Selective Update, and Experience Replay Buffer Flushing

This chapter introduces the OSDQN-RSF (One-State Deep Q-Network with epsilon Reset, Selective update, and Flushing replay buffer) method, which extends the standard DQN [12]–[14] algorithm with three key enhancements: (i) epsilon reset, (ii) selective parameter update, and (iii) flushing of the replay buffer. Compared with the classic DQN algorithm, the proposed OSDQN-RSF demonstrates a significant improvement in task scheduling accuracy on the same test set. Moreover, it achieves more efficient utilization of onboard satellite computing resources and reduces the associated computational cost.

1) *Motivation*: Despite its success, the standard Deep Q-Network (DQN) suffers from low sample efficiency, unstable training, and limited generalization in large continuous state spaces. It often converges poorly, gets trapped in local optima, and struggles when action rewards are nearly indistinguishable. To overcome these issues, we propose OSDQN-RSF (One-State DQN with epsilon reset, selective update, and flushing replay buffer), which improves generalization, accuracy, and convergence stability, achieving significantly better results than the baseline DQN on the test set.

2) *Epsilon Reset*: In the baseline DQN algorithm, the training process adopts an epsilon-greedy strategy. At the initial stage, the agent is assigned a relatively high epsilon value (e.g., 0.9), encouraging frequent random exploration. As training proceeds, epsilon gradually decreases toward nearly

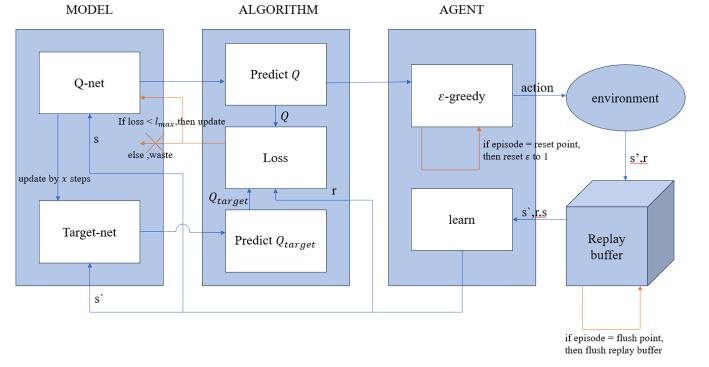


Fig. 3. Overview of the OSDQN-RSF algorithm workflow. Red arrows indicate modifications or differences compared with the original classical DQN algorithm.

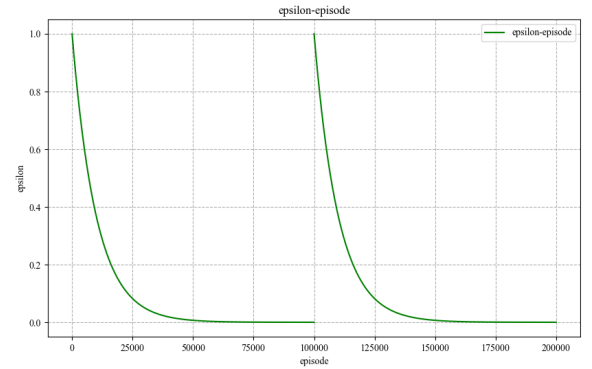


Fig. 4. Evolution of ϵ with respect to training episodes in the OSDQN-RSF algorithm (with reset point set to 100,000 and ϵ decay factor set to 0.9999 as an example).

zero, and the agent increasingly exploits its learned knowledge by selecting actions with the highest estimated Q-values to maximize cumulative rewards. At each timestep, the agent decides whether to explore or exploit based on the current epsilon value: with probability ϵ it selects a random action (exploration), and with probability $1 - \epsilon$ it selects the action with the maximum Q-value (exploitation). The Q-table or neural network parameters are then updated using the feedback from the environment to improve future action-value estimations.

The core idea of epsilon reset (as shown in Fig 4) is to restore epsilon to a relatively large value after a certain period of training, once the network has already learned an initial mapping from state space to action space. This mechanism encourages the addition of more exploratory actions to the replay buffer, thereby enhancing the network’s generalization ability in large, continuous state spaces. However, epsilon reset inevitably introduces a large number of random actions within a short period, which may temporarily degrade training performance and increase the number of training episodes required to reach the same level of effectiveness. To mitigate this drawback, we introduce the selective update mechanism, which selectively updates network parameters during this phase.

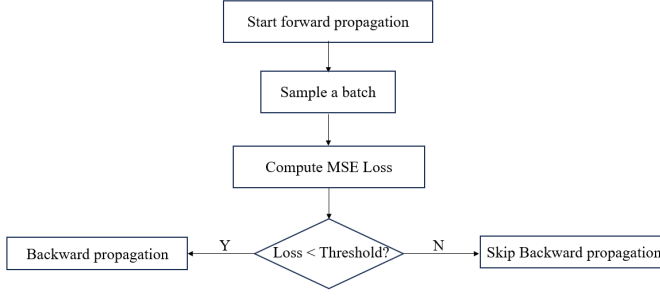


Fig. 5. The selective update mechanism in the OSDQN-RSF algorithm.

3) *Selective Update*: To mitigate the degradation in accuracy caused by the surge of random actions introduced through epsilon reset, and to better balance exploration and exploitation, we introduce the selective update mechanism.

In the baseline DQN algorithm, after sampling a mini-batch of state-action pairs from the replay buffer and computing the loss, the network parameters are updated directly via backpropagation. However, in the proposed DQN-RSF algorithm, once epsilon is reset, the replay buffer contains a large proportion of random actions. Consequently, the loss of a randomly sampled mini-batch may increase significantly. Learning from such batches is undesirable, as it implies that many state-action pairs in the batch deviate substantially from the mappings the network has already learned, with exploratory data dominating the update and impairing accuracy.

To address this issue, after epsilon reset we impose a loss threshold on each mini-batch (as shown in Fig 5): only those batches with a loss below the threshold are used for backpropagation. This strategy effectively reduces the negative impact of epsilon reset on network accuracy, ensuring that the model retains good generalization ability while maintaining high accuracy.

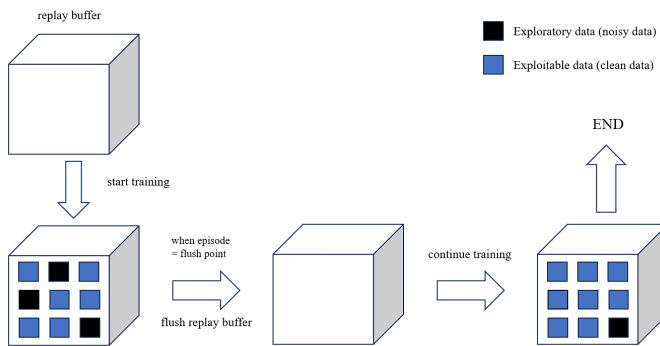


Fig. 6. Flushing replay buffer mechanism in the OSDQN-RSF algorithm.

4) *Flushing Replay Buffer*: The replay buffer is a key component of the baseline DQN algorithm, designed to store the agent's experiences during interaction with the environment. Each entry typically contains the current state, the action taken, the corresponding reward, and the next state. By sampling mini-batches of past experiences, DQN reduces data correlation, improves the stability and efficiency of learning,

and helps prevent overfitting. In essence, the replay buffer enables the agent to revisit and leverage past experiences more effectively, thereby optimizing decision-making in complex environments.

In the proposed DQN-RSF algorithm, after epsilon reset has been applied and subsequently decayed to a minimal value (typically 0.01), the network is expected to have already acquired sufficient generalization ability. At this stage, we introduce the flushing replay buffer mechanism, where the replay buffer is cleared (as shown in Fig 6). This is motivated by the fact that the buffer still contains a large proportion of random exploratory actions, which negatively impact accuracy. Following the flush, the replay buffer is refilled using the final minimal epsilon value: with probability ϵ the agent explores randomly, and with probability $1 - \epsilon$ it exploits the trained policy to select actions. This process greatly reduces the amount of noisy or uninformative data in the buffer and significantly improves accuracy once the network has attained adequate generalization capability. Compared with the baseline DQN, this mechanism provides a substantial performance enhancement.

5) *One State DQN*: In this study, the application scenario does not involve state transitions; that is, after each decision made by the reinforcement learning agent, the state vector remains unchanged. Under this condition, the computation of the target Q-value is defined as follows:

$$\begin{aligned} \text{where } Q_{\text{target}} &= r + \gamma \cdot f_{\text{target}}(\text{states}) \\ f_{\text{target}}(\text{states}) &= \max_{a'} Q_{\text{target}}(\text{states}, a') \end{aligned} \quad (12)$$

where Q_{target} represents the Q-value of the target net, r denotes the immediate reward, γ is the discount factor, and $f_{\text{target}}(\text{states})$ denotes the Q-value obtained by inputting the current state into the target network, and f represents the mapping performed by the target network on the state.

D. Summary

This chapter analyzes the limitations of the standard DQN algorithm during training and proposes the OSDQN-RSF (One-State DQN with epsilon Reset, Selective Update, and Flushing Replay Buffer) method. The proposed approach introduces three key improvements over the original DQN algorithm:

First, an epsilon reset mechanism is incorporated. Once the network has learned an initial mapping between the state and action spaces, epsilon is reset to a relatively large value to increase the number of exploratory actions, thereby enhancing the network's generalization capability in continuous high-dimensional state spaces.

Second, a selective update strategy is employed. By imposing a threshold on the loss of each mini-batch during backpropagation, the method prevents accuracy degradation caused by the influx of random exploratory actions.

Third, a flushing replay buffer mechanism is designed. After the network has achieved sufficient generalization, low-quality data in the replay buffer are removed, further improving the network's precision.

Together, these improvements allow OSDQN-RSF to achieve significantly higher task scheduling accuracy compared with the conventional DQN, while more efficiently utilizing satellite computing resources and reducing computational cost. Additionally, the network demonstrates enhanced convergence stability and superior performance in satellite task scheduling.

The practical performance of the proposed algorithm in scheduling tasks between large and small satellite models is detailed in the Experimental Results chapter.

The detailed pseudocode of the OSDQN-RSF algorithm is presented below:

Algorithm 1 OSDQN-RSF

```

1: Initialize replay memory  $\mathcal{D}$  to capacity  $N$ 
2: Initialize action-value network  $Q$ 
3: Initialize target network  $Q_{target}$  with parameters of  $Q$ 
4: for  $episode = 1$  to  $M$  do
5:   Initialize sequence  $s_1 = \{x_1\}$  and preprocessed sequence  $\phi_1 = \phi(s_1)$ 
6:   for  $t = 1$  to  $T$  do
7:     With probability  $\epsilon$ , select a random action  $a_t$ 
8:     Otherwise select  $a_t = \max_a Q(\phi(s_t), a)$ 
9:     Execute action  $a_t$  in emulator and observe reward  $r_t$  and next states vectors  $x_{t+1}$  (here  $x_{t+1} = x_t$ )
10:    Set  $s_{t+1} = s_t, a_t, x_{t+1}$  and preprocess  $\phi_{t+1} = \phi(s_{t+1})$ 
11:    Store transition  $(\phi_t, a_t, r_t, \phi'_{t+1})$  in  $\mathcal{D}$ 
12:    Sample random minibatch of transitions  $(\phi_j, a_j, r_j, \phi'_{j+1})$  from  $\mathcal{D}$ 
13:    Set target  $y_j = r + \gamma \max_{a'} Q(\phi_{j+1}, a')$ 
14:    if  $MSELoss(y, Q(\phi, a)) < x$  AND  $episode > reset\ point$  then
15:      Perform a gradient descent step on  $y_j - Q(\phi_j, a_j)$  according to  $MSELoss$ 
16:    end if
17:  end for
18:  if  $episode = reset\ point$  then
19:    Reset  $\epsilon$  to 1 and gradually decay to 0.01 for a second time
20:  end if
21:  if  $episode = flush\ point$  then
22:    Empty replay memory  $\mathcal{D}$ 
23:  end if
24:  Copy parameters of  $Q$  to  $Q_{target}$  every certain steps
25: end for

```

IV. EVALUATION

All neural networks in this experiment were trained on a ground-based server. The server is equipped with two 20-core Intel Xeon Gold 6133 CPUs running at 2.50 GHz, four NVIDIA GeForce RTX 4090 GPUs, and operates on Ubuntu 23.04. The implementation is developed in Python 3.8.20 using the deep learning framework PyTorch 2.2.0+cu121, managed under Anaconda 24.9.2.

A. Parameter Settings

TABLE III
PARAMETER SETTINGS OF DIMENSIONLESS COEFFICIENTS ρ AND WEIGHTS ω IN THE COST QUANTIFICATION MODEL

Reward function		
parameters	ρ (dimensionless)	ω (weight)
Small on-orbit model task		
Q (heat dissipation)	2	0.2
$FLOPs$ (computational load)	0.002	0.2
t_1 (task processing time)	0.002	0.1
t_2 (result transmission time)	0.002	0.1
$f(n)$ (device cost)	0.02	0.2
B (bandwidth occupancy)	2	0.2
Ground-based model task		
Q (heat dissipation)	2	1/3
t (result transmission time)	0.0015	1/3
B (bandwidth occupancy)	2	1/3
Preprocessed on orbit $\rightarrow$ ground model task		
Q_{pre} (data preprocessing heat dissipation)	2	0.2
Q_{trans} (data transmission heat dissipation)	2	0.2
t_{pre} (data preprocessing time)	0.002	0.2
t_{trans} (data transmission time)	0.002	0.2
$FLOPs$ (computational load)	0.002	0.2
B (bandwidth occupancy)	2	0.2

1) *Dimensionless Coefficients and Weight Assignments in Cost Quantification Modeling*: Since the cost quantification model requires the predefined configuration of dimensionless coefficients and weights, the parameter settings adopted in this study are provided here for reference. The detailed configurations are presented in Table III.

The above table presents the settings of dimensionless coefficients and weights used in the cost quantification model during the experiments in this study. For different scheduling strategies with specific emphases, the weights of certain cost components can be adjusted accordingly, so that they are given more consideration during scheduling. Table III shows the parameter settings used in the experiments of this study and is provided for reference only.

2) *Co-location Cost Function Setting*: In the previous sections, the co-location cost was denoted by $f(n)$. In the experiments, the actual function used is defined as:

$$f(n) = n^2 \quad (13)$$

That is, the co-location cost increases quadratically with the number of tasks n colocated on the same satellite board.

B. Generation of Training and Testing Datasets

1) *Training Dataset Generation*: Since this study employs a reinforcement learning algorithm, there is no need for pre-labeled data. Instead, training data can be generated in batches over the state space. Therefore, during the network training phase, the datasets are produced by randomly sampling task parameters within reasonable ranges.

The specific generation method is summarized in Table IV.

TABLE IV
RANDOM SAMPLING RANGES OF TRAINING NETWORK STATES AND
PARAMETER VALUES

Parameter	Description	Random Sampling Range (unit: parameter's own unit)
Task on orbit		
Q	Heat generation	(0,1000)
FLOPs	Computation	(0,1000)
t_1	Task processing time	(0,1000)
t_2	Result transmission time	(200,300)
n	Number of cores	(0,50)
B	Bandwidth usage	(0,1000)
Task on ground		
Q	Heat generation	(0,1000)
t	Result transmission time	(200,20000)
B	Bandwidth usage	(0,1000)
Preprocessed on-orbit $\rightarrow$ ground model task		
Q_{pre}	Data pre-processing heat	(0,1000)
Q_{trans}	Data transmission heat	(0,1000)
t_{pre}	Data pre-processing time	(0,1000)
t_{trans}	Data transmission time	(0,1000)
FLOPs	Computation	(200,20000)
B	Bandwidth usage	(0,1000)

2) *Testing Dataset Generation*: The generation of the testing dataset is similar to that of the training data, except that stricter bounds are applied to the variable ranges during generation. This ensures that the generated data values are more realistic and provide broad coverage across the state space for different scheduling strategies. Consequently, the testing dataset can accurately reflect the true performance of the trained network.

In this study, a total of 30,000 test instances are generated, with their distribution among the three scheduling strategies (in the order described in Chapter II) approximately following a 4:3:3 ratio. The testing dataset provides good coverage of the state space values. The detailed test data can be found in the project source files.

C. Analysis of Experimental Results

TABLE V
HYPERPARAMETER SETTINGS FOR TRAINING THE OSDQN-RSF
ALGORITHM

Hyperparameter	Value
Initial ϵ	1
ϵ_{decay}	0.9999
Learning rate	1×10^{-4}
Discount factor γ	0.9
Episodes	350,000 – 600,000
Replay buffer size	1,000,000 (deque)
Batch size	256
Loss maximum	15
Reset point	100,000
Flush point	200,000

In this section, the network architecture trained by the OSDQN-RSF algorithm is the same as illustrated in Section III-B. After extensive hyperparameter tuning, the configuration that achieved the best performance was selected. The final training parameters are as follows: the initial value of ϵ was set to 1 with an ϵ_{decay} of 0.9999; the learning rate was 1×10^{-4} ; the discount factor γ was 0.9; the number of episodes was intermittently sampled between 350,000 and

600,000; the replay buffer size was implemented as a deque with a maximum length of 1,000,000; the batch size was 256; the loss maximum was constrained to 15; and the reset point and flush point were set to 100,000 and 200,000, respectively.

The model accuracy reported in Fig 7 was obtained using the self-constructed dataset introduced in Section IV-B. As illustrated in Fig 7, under the above hyperparameter settings, the accuracy of the DQN-RSF algorithm remains relatively stable when the number of episodes ranges between 350,000 and 500,000, indicating that DQN-RSF is not highly sensitive to the number of training episodes and tends to converge easily. The best training run achieved a test set accuracy of 95.3%.

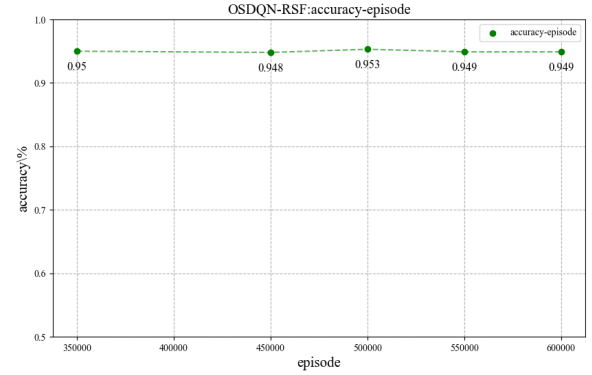


Fig. 7. Test set accuracy of the OSDQN-RSF algorithm under different numbers of training episodes.

D. Comparative Analysis of Cost Savings

TABLE VI
AVERAGE COSTS OF DIFFERENT ALGORITHMS ON TEST SET

	Q/J	t/ms	FLOPs/MFLOPs	B/Mbps
OSDQN-RSF	539.712	2679.510	419.392	338.674
Large Model	570.540	11442.742	0	616.702
Small Model	791.493	1041.580	791.480	572.935

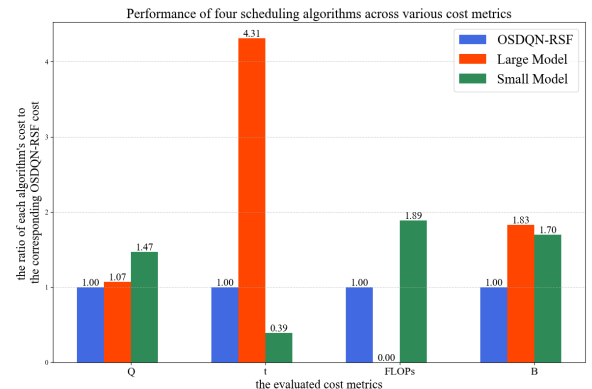


Fig. 8. Performance of different scheduling strategies across various cost metrics.

In the comparative analysis of cost savings, the OSDQN-RSF model with the highest accuracy was selected. The evaluation was conducted on 30,000 test samples, where the average values of multiple cost metrics were calculated, including onboard heat generation, processing latency, satellite-to-ground bandwidth occupancy, and onboard computational workload.

The results obtained from the OSDQN-RSF algorithm were compared against two baseline scenarios: no scheduling and processing tasks exclusively (i) in-orbit or (ii) on the ground. The comparative outcomes are summarized in Table VI and illustrated in Fig 8.

In Table VI, the terms *Large Model* and *Small Model* refer to non-reinforcement learning scheduling strategies, where all tasks are directly allocated to either the ground-based large model or the onboard small model for computation without any evaluation. In Figure 8, the FLOPs cost of assigning all tasks to the ground-based large model is zero because the satellite does not undertake any computation in this case, and all required data are transmitted to the ground for processing. Since the computational cost on the ground is not the focus of this study, it is reasonably ignored. However, this strategy of transmitting all raw task data to the ground without any preprocessing also results in a significantly higher time cost compared with other scheduling strategies.

By examining the figures, it is evident that, aside from the aforementioned special case, the application of reinforcement learning algorithms demonstrates substantial improvements in various cost metrics compared to strategies that directly assign tasks to the ground-based large model or the onboard small model.

E. Summary

Based on the evaluation conducted on a test set containing 30,000 samples, the statistical analysis of the OSDQN-RSF algorithm demonstrates its effectiveness in reducing overall cost consumption. The proposed satellite task scheduling algorithm, built upon OSDQN-RSF, significantly decreases the utilization of satellite computing resources, optimizes task placement, and achieves a balance between task completion efficiency and operational cost.

Furthermore, the OSDQN-RSF algorithm enhances the training process of classic DQN algorithm, exhibiting superior performance in the satellite task scheduling scenario studied in this work. These findings also suggest its potential applicability to a broader range of reinforcement learning tasks beyond the present context.

V. RELATED WORK

Despite significant progress in satellite task scheduling, the field still faces severe energy and computational resource limitations: the restricted onboard computing capacity makes it challenging to efficiently utilize limited resources for diverse mission requirements [15].

Emiliano Casalicchio et al. from the University of Rome proposed a system model that formulates the scheduling problem as an optimization task, aiming to minimize the

average task response time under constraints of resource availability and task deadlines, and demonstrated through simulation that their decentralized algorithm outperforms centralized and distributed baselines in scalability and availability [16]. Junhong Kim et al. from KAIST introduced a framework for scheduling observation and download tasks of multiple agile satellites by developing a mixed integer linear programming (MILP) model that incorporates practical factors such as attitude maneuvering, onboard data storage, and stereo imaging, and further proposed a heuristic based on time-window pruning [17] to approximate near-optimal solutions, validated via case studies [18]. Shen Xin's group at the State Key Laboratory of Information Engineering in Surveying, Mapping and Remote Sensing developed a SAR satellite strip mosaic task planning model that jointly optimizes orbit resources and imaging sequences under payload power consumption constraints, and proposed an improved particle swarm optimization algorithm (LSDE-PSO) combining local search and differential evolution to enhance efficiency in solving complex large-scale models [19]. The same group also proposed a spiking neural network with PPO-based strategy optimization (SNN-PPO) for multi-target imaging task scheduling, leveraging the temporal characteristics of SNN to capture task dependencies, reducing attitude maneuvering costs through a task-switching time matrix, and achieving global optimal task sequencing under complex conditions [20]. Weisen Liu et al. from Tsinghua University presented Phoenix, an energy-efficient scheduling framework for Space Edge Computing (SEC [21]) networks, which formulates the SEC Battery Energy Optimization (SBEO) problem to minimize average battery consumption while ensuring task deadlines [22]. Qiangqiang Jiang et al. from Harbin Institute of Technology proposed a scheduling model for complex orbital remote sensing image processing tasks in heterogeneous multiprocessor systems (HMPS [23]), achieving approximately 51% reduction in makespan and 54% reduction in TEC, with a $2.2\times$ improvement in energy efficiency and up to 2.56×10^5 pixels/W/s processing capacity [24]. The same team further introduced SECORS, a resource scheduling technique with a dedicated SEC architecture for remote sensing, along with an end-to-end algorithm (SEC-MPH), which reduces onboard computing time and energy consumption by 68.87% and 66.60%, respectively, while tripling energy efficiency and reaching a processing capability of 548 pixels/W/ms [25].

VI. CONCLUSION

This study proposes a multi-dimensional cost evaluation model tailored to the characteristics of satellite computing, with a particular focus on key cost factors including thermal generation, latency, power consumption, and task co-location. The design of this model provides an effective tool for the quantitative analysis of the operational costs of different types of tasks in satellite computing environments, and lays a foundation for optimizing task allocation strategies as well as making rational decisions on whether to offload tasks to ground nodes or to satellite systems.

Building on the proposed quantitative cost modeling, this study further develops a scheduling algorithm between large-

scale remote sensing models on the ground and small-scale remote sensing models onboard satellites, based on the Deep Q-Network (DQN) in reinforcement learning. Improvements upon the standard DQN framework lead to the proposed **OSDQN-RSF** algorithm. The OSDQN-RSF algorithm demonstrates excellent performance in the scheduling task, offering a feasible approach for task allocation between ground-based large models and satellite-based small models.

Experimental results indicate that the scheduling strategy based on the OSDQN-RSF algorithm reduces the consumption of satellite-side computational costs to one-half to one-quarter of those incurred when tasks are computed solely on ground-based large models or solely on satellite-based small models. Furthermore, the proposed OSDQN-RSF algorithm optimizes the model training process beyond the standard DQN, achieving an accuracy of **95.3%** on a self-constructed validation set. This enables more efficient scheduling of computational tasks between ground-based large models and satellite-based small models, thereby saving satellite computational resources and reducing various costs. In addition, the OSDQN-RSF algorithm shows potential for adaptation and application to other reinforcement learning problems.

REFERENCES

- [1] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 779–788.
- [2] R. Girshick, "Fast r-cnn," in *Proceedings of the IEEE international conference on computer vision*, pp. 1440–1448.
- [3] C. M. Ward, J. Harguess, B. Crabb, and S. Parameswaran, "Image quality assessment for determining efficacy and limitations of super-resolution convolutional neural network (srcnn)," in *Applications of Digital Image Processing XL*, vol. 10396, pp. 19–30, SPIE.
- [4] X. Wang, K. Yu, S. Wu, J. Gu, Y. Liu, C. Dong, Y. Qiao, and C. Change Loy, "Esrgan: Enhanced super-resolution generative adversarial networks," in *Proceedings of the European conference on computer vision (ECCV) workshops*, pp. 0–0.
- [5] T. Salimans, I. Goodfellow, W. Zaremba, V. Cheung, A. Radford, and X. Chen, "Improved techniques for training gans," *Advances in neural information processing systems*, vol. 29, 2016.
- [6] T. Karras, S. Laine, M. Aittala, J. Hellsten, J. Lehtinen, and T. Aila, "Analyzing and improving the image quality of stylegan," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pp. 8110–8119.
- [7] X. Zhang, T. Zhang, G. Wang, P. Zhu, X. Tang, X. Jia, and L. Jiao, "Remote sensing object detection meets deep learning: A metareview of challenges and advances," *IEEE Geoscience and Remote Sensing Magazine*, vol. 11, no. 4, pp. 8–44, 2023.
- [8] S. Wang and Q. Li, "Satellite computing: Vision and challenges," *IEEE Internet of Things Journal*, vol. 10, no. 24, pp. 22514–22529, 2023.
- [9] L. P. Kaelbling, M. L. Littman, and A. W. Moore, "Reinforcement learning: A survey," *Journal of artificial intelligence research*, vol. 4, pp. 237–285, 1996.
- [10] M. Elbuluk, A. Hammoud, and R. Patterson, "Power electronic components, circuits and systems for deep space missions," in *2005 IEEE 36th Power Electronics Specialists Conference*, pp. 1156–1162, IEEE.
- [11] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: a simple way to prevent neural networks from overfitting," *The journal of machine learning research*, vol. 15, no. 1, pp. 1929–1958, 2014.
- [12] R. Bellman, "Dynamic programming," *science*, vol. 153, no. 3731, pp. 34–37, 1966.
- [13] G. Tesauro, "Temporal difference learning and td-gammon," *Communications of the ACM*, vol. 38, no. 3, pp. 58–68, 1995.
- [14] C. J. Watkins and P. Dayan, "Q-learning," *Machine learning*, vol. 8, pp. 279–292, 1992.
- [15] J. Wu, Y. Chen, Y. He, L. Xing, and Y. Hu, "Survey on autonomous task scheduling technology for earth observation satellites," *Journal of Systems Engineering and Electronics*, vol. 33, no. 6, pp. 1176–1189, 2022.
- [16] E. Casalicchio and D. Magliarisi, "Decentralized task scheduling in satellite edge computing," in *2024 9th International Conference on Fog and Mobile Edge Computing (FMEC)*, pp. 154–161, IEEE.
- [17] P. Molchanov, S. Tyree, T. Karras, T. Aila, and J. Kautz, "Pruning convolutional neural networks for resource efficient inference," *arXiv preprint arXiv:1611.06440*, 2016.
- [18] J. Kim, J. Ahn, H.-L. Choi, and D.-H. Cho, "Task scheduling of agile satellites with transition time and stereoscopic imaging constraints," *Journal of Aerospace Information Systems*, vol. 17, no. 6, pp. 285–293, 2020.
- [19] X. Shen, Z. Lu, L. Li, Y. Chen, X. Li, J. Wang, and W. Yao, "Multi-strip stitching imaging mission planning method for sar satellites regional mapping considering on-board energy consumption," *IEEE Transactions on Geoscience and Remote Sensing*, 2024.
- [20] W. Yao, X. Shen, G. Zhang, Z. Lu, J. Wang, Y. Song, and Z. Li, "A spiking neural network based proximal policy optimization method for multi-point imaging mission scheduling of earth observation satellite," *Swarm and Evolutionary Computation*, vol. 94, p. 101867, 2025.
- [21] Y. Qiu, J. Niu, X. Zhu, K. Zhu, Y. Yao, B. Ren, and T. Ren, "Mobile edge computing in space-air-ground integrated networks: Architectures, key technologies and challenges," *Journal of Sensor and Actuator Networks*, vol. 11, no. 4, p. 57, 2022.
- [22] W. Liu, Z. Lai, Q. Wu, H. Li, Q. Zhang, Z. Li, Y. Li, and J. Liu, "In-orbit processing or not? sunlight-aware task scheduling for energy-efficient space edge computing networks," in *IEEE INFOCOM 2024-IEEE Conference on Computer Communications*, pp. 881–890, IEEE.
- [23] J. Chen, P. Han, Y. Zhang, T. You, and P. Zheng, "Scheduling energy consumption-constrained workflows in heterogeneous multi-processor embedded systems," *Journal of Systems Architecture*, vol. 142, p. 102938, 2023.
- [24] Q. Jiang, H. Wang, Q. Kong, Y. Zhang, and B. Chen, "On-orbit remote sensing image processing complex task scheduling model based on heterogeneous multiprocessor," *IEEE Transactions on Geoscience and Remote Sensing*, vol. 61, pp. 1–18, 2023.
- [25] Q. Jiang, L. Zheng, Y. Zhou, H. Liu, Q. Kong, Y. Zhang, and B. Chen, "Efficient on-orbit remote sensing imagery processing via satellite edge computing resource scheduling optimization," *IEEE Transactions on Geoscience and Remote Sensing*, 2025.